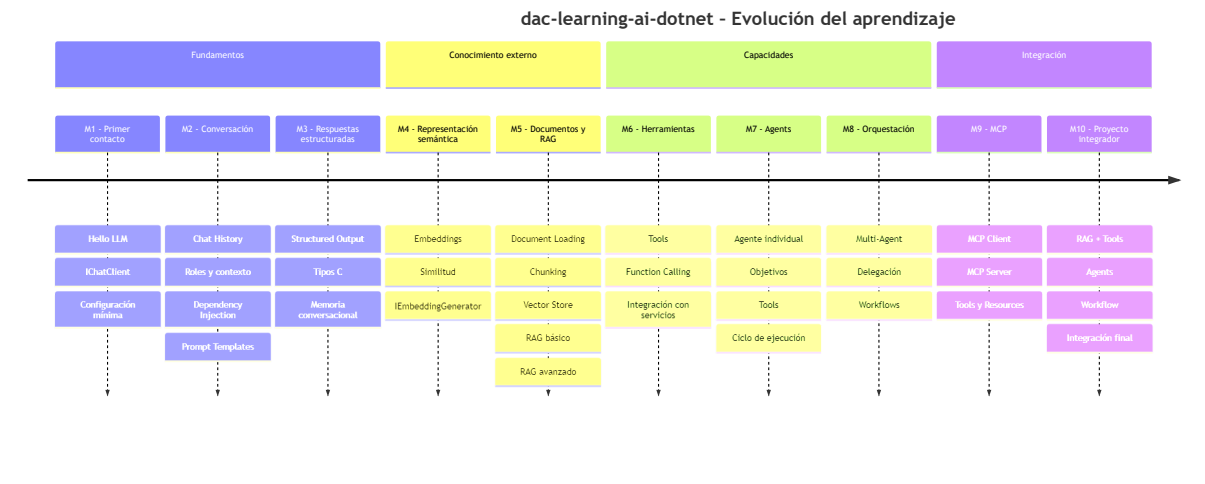


DAC Learning AI .NET – Roadmap

Proyecto: dac-learning-ai-dotnet

Roadmap de aprendizaje incremental para construir aplicaciones con IA generativa en .NET, manteniendo una correspondencia conceptual con `dac-learning-langchain4j`.

Visión de evolución



Estado general

Hito	Tema	Labs	Estado
M1	Primer contacto con LLM	01	✅ Implementado
M2	Conversación, DI y prompts	02–04	⌚ Pendiente
M3	Structured Output y memoria	05–06	⌚ Pendiente
M4	Embeddings	07	⌚ Pendiente
M5	Documentos y RAG	08–10	⌚ Pendiente
M6	Tools y Function Calling	11–12	⌚ Pendiente
M7	Agents	13	⌚ Pendiente
M8	Multi-Agent y Workflows	14–15	⌚ Pendiente

Hito	Tema	Labs	Estado
M9	MCP	16	⌚ Pendiente
M10	Proyecto integrador	17	⌚ Pendiente

M1 — Primer contacto con un LLM

Objetivo: realizar una interacción mínima con un modelo de lenguaje desde .NET.

Estado:  Implementado

Lab 01 — Hello LLM

#	Estado	Alcance
1.1		Crear una aplicación Console mínima
1.2		Configurar acceso al modelo mediante variable de entorno
1.3		Utilizar <code>IChatClient</code> como abstracción principal
1.4		Enviar un prompt simple
1.5		Mostrar la respuesta del modelo
1.6		Documentar ejecución desde VS Code Insiders

Conceptos incorporados

- cliente de chat;
- prompt;
- respuesta;
- configuración externa;
- separación mínima respecto del proveedor.

Fuera de alcance

- historial;
- memoria;
- Dependency Injection;
- Semantic Kernel;

- tools;
- RAG.

Resultado esperado

```

Prompt
↓
IChatClient
↓
Modelo
↓
Respuesta

```

M2 — Conversación, servicios y prompts

Objetivo: pasar de una llamada aislada a una pequeña aplicación conversacional correctamente estructurada.

Estado: ⌚ Pendiente

Lab 02 — Chat History

#	Estado	Alcance
2.1	⌚	Introducir una colección de mensajes
2.2	⌚	Diferenciar roles <code>system</code> , <code>user</code> y <code>assistant</code>
2.3	⌚	Mantener contexto entre preguntas
2.4	⌚	Mostrar la diferencia entre llamada aislada e historial

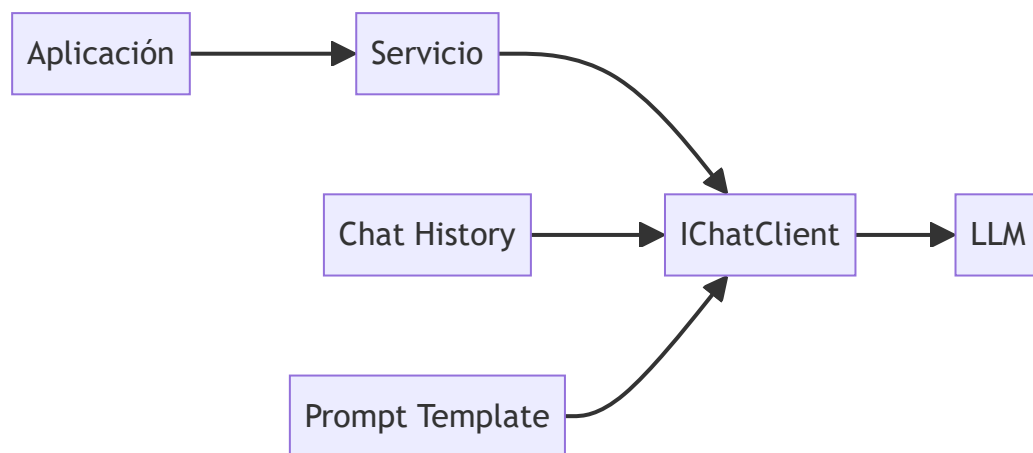
Lab 03 — Services y Dependency Injection

#	Estado	Alcance
3.1	⌚	Incorporar el contenedor de DI de .NET
3.2	⌚	Registrar <code>IChatClient</code>
3.3	⌚	Crear un servicio consumidor
3.4	⌚	Separar configuración, infraestructura y lógica de aplicación

Lab 04 — Prompt Templates

#	Estado	Alcance
4.1	⌚	Crear prompts parametrizados
4.2	⌚	Separar contenido del prompt del código
4.3	⌚	Reutilizar una plantilla con distintos valores
4.4	⌚	Incorporar instrucciones de sistema cuando corresponda

Resultado esperado



M3 — Structured Output y memoria

Objetivo: dejar de tratar todas las respuestas del modelo como texto libre.

Estado: ⌚ Pendiente

Lab 05 — Structured Output

#	Estado	Alcance
5.1	⌚	Solicitar una respuesta estructurada
5.2	⌚	Utilizar JSON como contrato
5.3	⌚	Mapear el resultado a <code>class</code> o <code>record</code> C#
5.4	⌚	Manejar respuestas inválidas

#	Estado	Alcance
5.5		Validar datos mínimos de salida

Lab 06 — Memoria conversacional avanzada

#	Estado	Alcance
6.1		Diferenciar historial y memoria
6.2		Limitar la ventana conversacional
6.3		Evaluar estrategias de resumen
6.4		Introducir persistencia básica de contexto

Resultado esperado

```

LLM
↓
Structured Output
↓
Tipo C#
↓
Aplicación

```

M4 — Embeddings

Objetivo: comprender cómo representar texto de forma semántica y comparar significado.

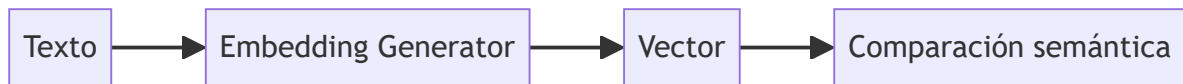
Estado:  Pendiente

Lab 07 — Embeddings

#	Estado	Alcance
7.1		Generar embeddings
7.2		Utilizar <code>IEmbeddingGenerator</code>
7.3		Comparar vectores
7.4		Introducir similitud semántica

#	Estado	Alcance
7.5		Diferenciar búsqueda textual de búsqueda semántica

Flujo



M5 — Documentos y RAG

Objetivo: incorporar conocimiento externo y construir un pipeline RAG completo.

Estado:  Pendiente

Lab 08 — Document Loading

#	Estado	Alcance
8.1		Leer documentos
8.2		Extraer texto
8.3		Normalizar contenido
8.4		Aplicar chunking
8.5		Asociar metadatos

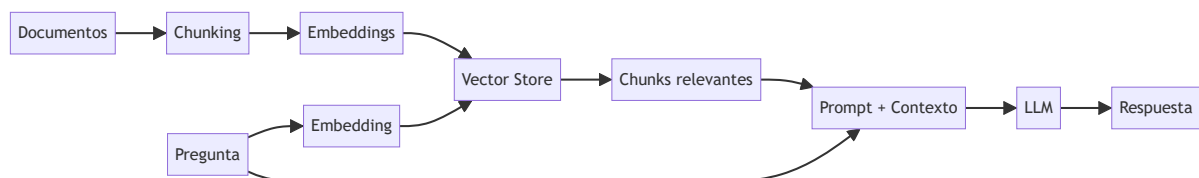
Lab 09 — RAG básico

#	Estado	Alcance
9.1		Generar embeddings de documentos
9.2		Almacenar vectores
9.3		Generar embedding de una consulta
9.4		Recuperar chunks relevantes
9.5		Construir prompt con contexto
9.6		Generar respuesta basada en contexto recuperado

Lab 10 — RAG avanzado

#	Estado	Alcance
10.1	⌚	Comparar estrategias de chunking
10.2	⌚	Utilizar top-k
10.3	⌚	Aplicar score mínimo
10.4	⌚	Incorporar filtros por metadata
10.5	⌚	Introducir ranking o re-ranking
10.6	⌚	Evaluar calidad de recuperación

Arquitectura conceptual



M6 — Tools y Function Calling

Objetivo: permitir que el modelo utilice capacidades externas controladas por la aplicación.

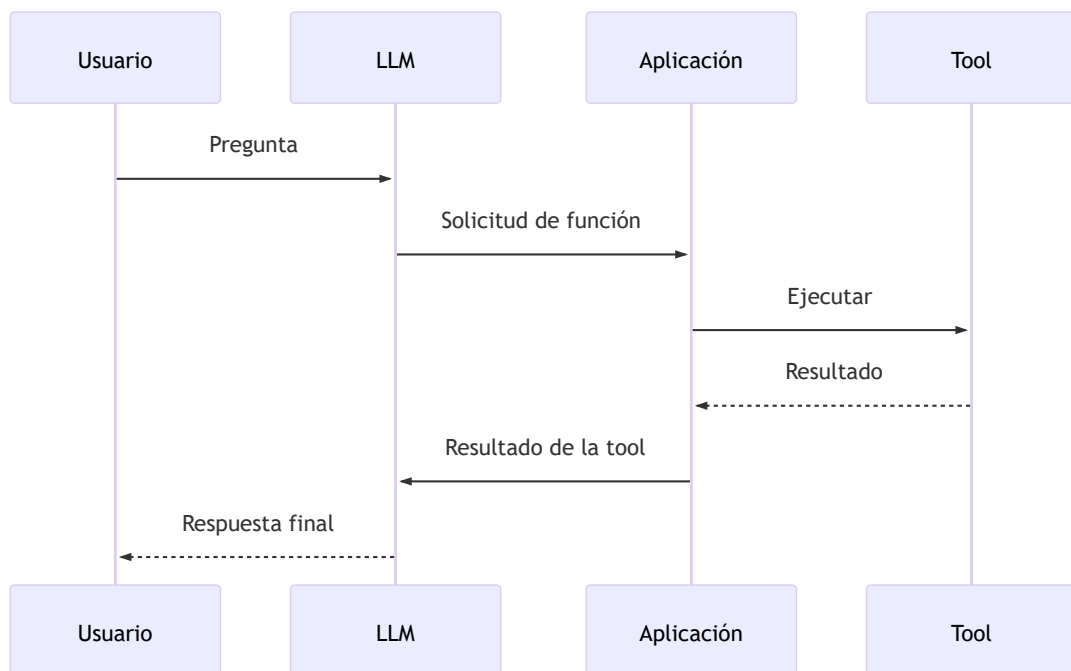
Estado: ⌚ Pendiente

Lab 11 — Tools

#	Estado	Alcance
11.1	⌚	Definir una función como herramienta
11.2	⌚	Describir su finalidad
11.3	⌚	Definir parámetros
11.4	⌚	Exponer servicios locales como tools

Lab 12 — Function Calling

#	Estado	Alcance
12.1	⌚	Permitir que el modelo seleccione una función
12.2	⌚	Interpretar argumentos
12.3	⌚	Ejecutar la función
12.4	⌚	Devolver el resultado al modelo
12.5	⌚	Generar la respuesta final



M7 — Agents

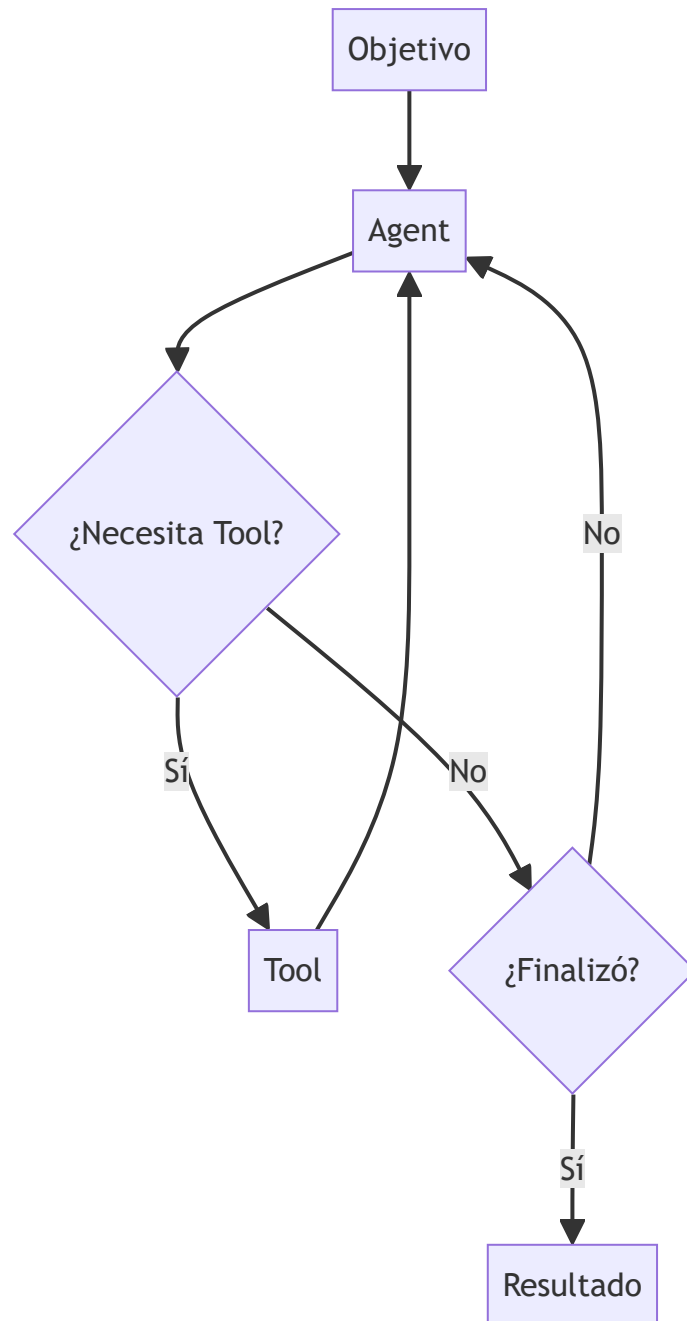
Objetivo: introducir comportamiento agéntico de forma controlada.

Estado: ⌚ Pendiente

Lab 13 — Agents

#	Estado	Alcance
13.1	⌚	Definir objetivo e instrucciones
13.2	⌚	Asignar tools al agente
13.3	⌚	Ejecutar un ciclo de decisión y acción

#	Estado	Alcance
13.4	⌚	Establecer condición de finalización
13.5	⌚	Incorporar límites de ejecución
13.6	⌚	Observar las acciones realizadas



M8 — Multi-Agent y Workflows

Objetivo: separar responsabilidades y coordinar múltiples participantes.

Estado: ⌚ Pendiente

Lab 14 — Multi-Agent

#	Estado	Alcance
14.1	⌚	Crear agentes especializados
14.2	⌚	Delegar tareas
14.3	⌚	Compartir o transferir contexto
14.4	⌚	Consolidar resultados

Lab 15 — Workflows

#	Estado	Alcance
15.1	⌚	Definir pasos explícitos
15.2	⌚	Incorporar estado
15.3	⌚	Incorporar bifurcaciones
15.4	⌚	Ejecutar pasos secuenciales
15.5	⌚	Evaluar ejecución paralela
15.6	⌚	Integrar agents y funciones dentro del flujo

Diferencia conceptual

Agent

decide parte del camino

Workflow

define explícitamente el camino

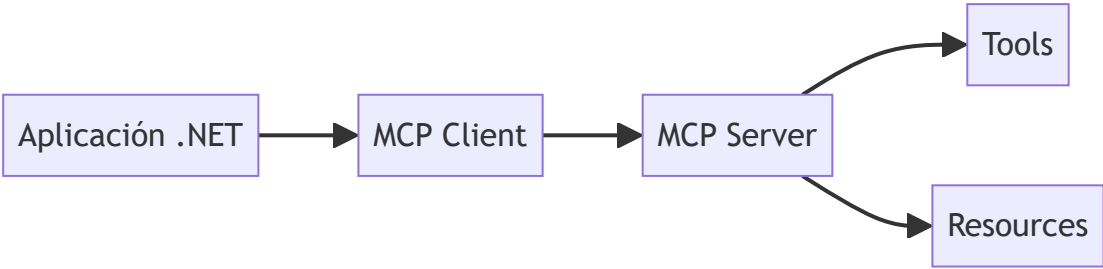
M9 — Model Context Protocol

Objetivo: comprender cómo conectar herramientas y recursos mediante un protocolo estándar.

Estado: 🕒 Pendiente

Lab 16 — MCP

#	Estado	Alcance
16.1	🕒	Comprender la arquitectura MCP
16.2	🕒	Implementar o utilizar un MCP Client
16.3	🕒	Conectarse a un MCP Server
16.4	🕒	Descubrir tools
16.5	🕒	Consumir resources
16.6	🕒	Ejecutar una tool remota



M10 — Proyecto integrador

Objetivo: integrar los conceptos estudiados en una solución pequeña y comprensible.

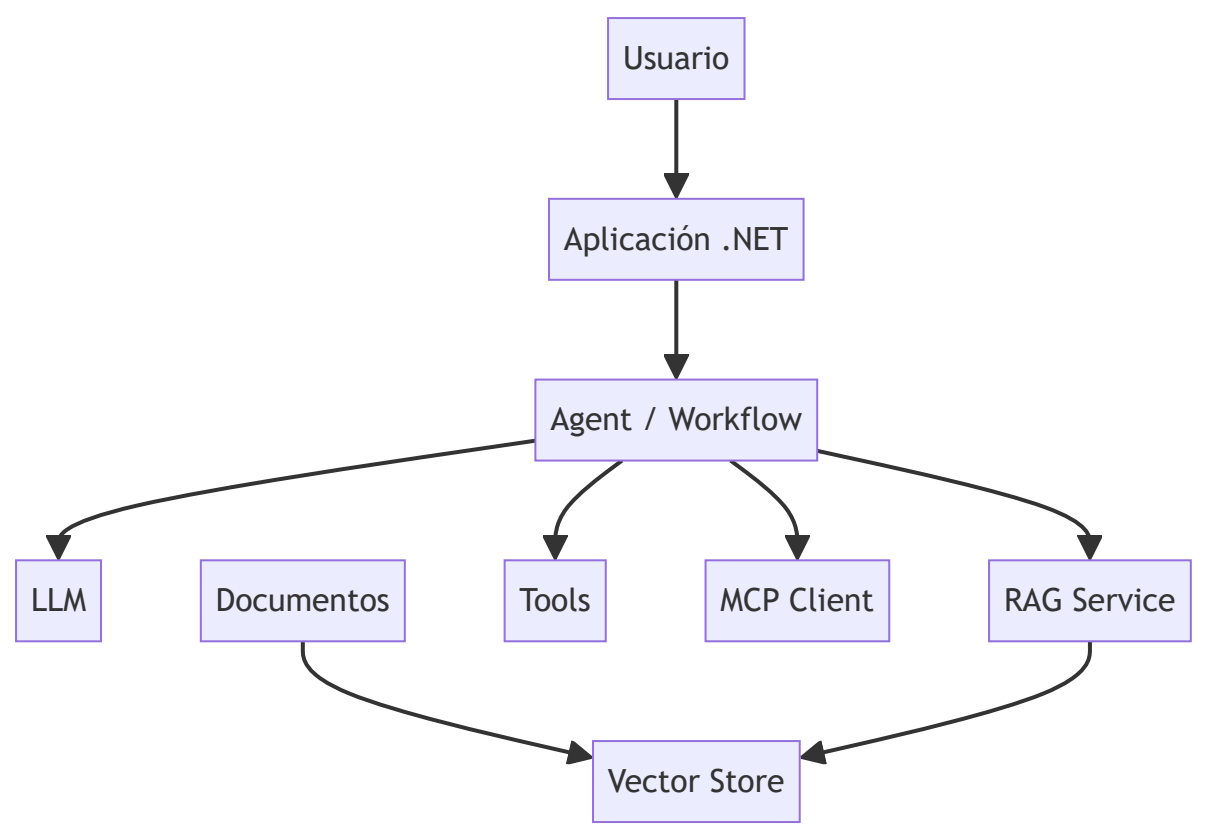
Estado: 🕒 Pendiente

Lab 17 — Proyecto final

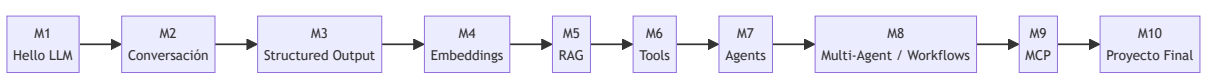
#	Estado	Alcance
17.1	🕒	Utilizar <code>IChatClient</code>
17.2	🕒	Configurar servicios mediante DI
17.3	🕒	Utilizar prompts reutilizables
17.4	🕒	Incorporar structured output
17.5	🕒	Incorporar documentos y embeddings

#	Estado	Alcance
17.6	⌚	Implementar un flujo RAG
17.7	⌚	Incorporar al menos una tool
17.8	⌚	Utilizar function calling
17.9	⌚	Incorporar al menos un agent
17.10	⌚	Orquestar mediante un workflow
17.11	⌚	Evaluar integración MCP

Arquitectura orientativa



Dependencias entre hitos



Criterio de finalización de un laboratorio

Un laboratorio se considera completo cuando dispone de:

- ☐ objetivo claro;
- ☐ explicación conceptual;
- ☐ código mínimo ejecutable;
- ☐ instrucciones de ejecución;
- ☐ resultado esperado;
- ☐ README propio;
- ☐ explicación de las piezas principales;
- ☐ comparación conceptual con LangChain4j, cuando aporte valor;
- ☐ commit estable en Git.

Posibles extensiones futuras

Estas líneas quedan deliberadamente fuera del roadmap principal:

- modelos locales con Ollama;
- Azure OpenAI;
- observabilidad;
- evaluación automática de respuestas;
- evaluación de RAG;
- guardrails;
- persistencia avanzada;
- bases vectoriales externas;
- multimodalidad;
- audio;
- visión;
- GraphRAG;
- integración con IDEs.

Se incorporarán como laboratorios adicionales solamente si aportan valor al recorrido principal.